

---

# 学管系统

## 详细设计方案

V2.1

什么都队开发团队  
2026 年 4 月 20 日

# 版本控制

| 版本   | 修订日期      | 描述    | 修订者  | 审批人  | 审批日期      |
|------|-----------|-------|------|------|-----------|
| V1.1 | 2026/3/25 | 第一版草稿 | 诸葛瑞乐 | 孙志瑞  | 2026/3/26 |
| V2.0 | 2026/4/5  | 补充、修改 | 孙志瑞  | 诸葛瑞乐 | 2026/4/6  |
| V2.1 | 2026/4/18 | 补充、修改 | 孙志瑞  | 诸葛瑞乐 | 2026/4/19 |
|      |           |       |      |      |           |

---

## 目录

|                  |    |
|------------------|----|
| 第一章 软件架构.....    | 1  |
| 1.1 软件设计概述.....  | 1  |
| 1.2 软件架构.....    | 1  |
| 第二章 数据获取.....    | 1  |
| 2.1 数据来源.....    | 3  |
| 2.2 数据清洗.....    | 3  |
| 2.3 数据集成.....    | 4  |
| 2.4 数据变换.....    | 4  |
| 2.5 特征选择.....    | 4  |
| 2.6 数据划分.....    | 5  |
| 第三章 函数与模型创建..... | 5  |
| 3.1 函数的创建.....   | 5  |
| 3.2 模型的创建.....   | 6  |
| 第四章 项目测试.....    | 8  |
| 4.1 测试方法.....    | 8  |
| 4.2 测试过程.....    | 8  |
| 4.3 测试结论.....    | 9  |
| 第五章 算法接口.....    | 9  |
| 5.1 接口代码.....    | 9  |
| 5.2 代码说明.....    | 11 |
| 第六章 附件.....      | 13 |

---

# 第一章 软件架构

## 1.1 软件设计概述

此 1. 需求背景与目标 本教务教学管理系统（以下简称“学管系统”）旨在解决高校教学过程中选课信息滞后、成绩录入繁琐及数据安全性难以保障等痛点。系统核心目标是构建一个支持多角色（学生、教师、管理员）协同工作的数字化平台。

- **学生端**：实现自主选课、修读进度查看及历史成绩查询。
- **教师端**：实现对所授课程的教学激活、学生修读名单管理及课程成绩的闭环录入。
- **管理端**：负责基础档案维护、课程发布及全局权限调控。

2. 设计思路 系统基于“业务解耦”与“安全隔离”的核心原则进行设计：

- **角色驱动架构**：采用基于角色的访问控制（RBAC）模型，确保不同角色用户仅能访问其职权范围内的功能菜单。
- **模块垂直拆分**：为了降低系统耦合度，将“学生选课”与“教师评改”从业务逻辑层到表现层进行彻底剥离，独立维护各自的 API 路由与逻辑实现。
- **数据安全增强**：在涉及成绩与课程的核心接口中，引入服务端身份二次校验机制，防止通过前端漏洞进行的非法越权操作（如学生篡改成绩、教师操作他人课程）。

## 1.2 软件架构

本系统采用主流的 B/S（Browser/Server）架构，基于若依（RuoYi）全栈开发框架构建，整体呈现标准的前后端分离三层架构模式。

1. 技术栈构成

- **前端展示层**：基于 Vue.js 2.x 框架与 Element UI 组件库，利

用动态路由技术实现根据登录角色展示差异化菜单。

- **后端业务层：**基于 Spring Boot 2.x 框架，核心业务逻辑封装在 Service 接口及其实现类中。
- **持久化数据层：**采用 MySQL 8.0 作为关系型数据库，利用 MyBatis 框架处理复杂的连表查询（如学生档案与选课记录的关联）。
- **缓存与安全：**利用 Redis 存储 Token 凭证与权限缓存，通过 Spring Security 确保接口调用的安全性。

## 2. 软件逻辑架构（结构说明）

- **表示层 (Presentation Layer)：**
  - **教师管理模块：**包含“成绩批改”专属视图，支持主子表交互（课程列表 -> 学生名单）。
  - **学生业务模块：**包含“学生选课”视图，支持课程实时搜索与状态追踪。
- **业务逻辑层 (Business Logic Layer)：**
  - **TeacherGradeService：**处理开课校验（如人数检查）、成绩录入合规性检查（0-100 分约束）及学分自动结算逻辑。
  - **StudentCourseService：**处理选课冲突校验、选课容量控制及状态流转。
- **数据访问层 (Data Access Layer)：**
  - 通过 BizTeacherGradeMapper 实现复杂的多表关联 SQL，确保教师能同时获取学生的基本档案信息（姓名、学院等）与选课状态数据。

## 3. 数据隔离机制说明

本系统在架构层面引入了“数据过滤拦截器”。在执行课程列表查询时，系统会自动抓取当前登录用户的唯一标识，并将其注入 SQL 查询条件的 teacher\_no 或 user\_id 字段中。这种设计确保了在物理层面上，教师只能查询和操作自己名下的教学数据，实现了极高的数据安全性。

## 第二章 数据获取

主要介绍数据获取的方式、方法。

### 2.1 数据来源

系统的数据来源主要分为以下三类：

1. **基础档案数据**：由管理员通过后台管理模块统一导入或录入，包括学生基本信息（学号、姓名、专业）、教师基本信息及课程元数据（课程名、学分、限选人数）。
2. **业务交互数据**：在系统运行过程中，由学生执行“选课”操作产生。该数据持久化存储于 `biz_student_course` 中间表，建立学生与课程的动态关联。
3. **评价衍生数据**：由教师在教学评估阶段录入的成绩数据，并由系统根据业务逻辑自动计算生成的衍生数据（如：是否及格、已获学分等）。

### 2.2 数据清洗

在数据进入数据库之前，系统通过后端 `Service` 层进行严格的“清洗”与校验，以确保数据的准确性与完整性：

- **重复值处理**：在学生选课时，系统会自动检索 `biz_student_course` 表，校验是否存在“同学生同课程”的重复选课记录，防止脏数据产生。
- **异常值拦截**：在成绩录入模块，系统会对 `score` 字段进行区间校验。若输入值不在 `[0, 100]` 范围内，后端将抛出 `ServiceException` 并拦截入库请求。
- **约束性检查**：在开课阶段，系统会检索当前课程的已选人数，若人数为 0 则判定为“异常状态”，拦截开课指令并给出业务提示。

## 2.3 数据集成

由于业务需求涉及到跨表展示（例如：老师在批改成绩时需要同时看到学生的姓名和课程名称），系统采用了基于 MyBatis 的多表左连接（LEFT JOIN）技术进行数据集成：

- **关联路径：**以 `biz_student_course` 为核心主表，通过 `student_no` 关联 `biz_student_profile`（获取姓名、学院），通过 `course_id` 关联 `biz_course`（获取课程名、总学分）。
- **集成结果：**将分散在三个物理表中的字段集成到 Java 实体类 `BizStudentCourse` 的扩展属性中，实现数据的统一视图展示。

## 2.4 数据变换

为了提升数据的可读性与业务价值，系统在展示层和业务层进行了数据变换：

- **状态映射：**将数据库中存储的数字状态（`learn_status`）变换为具有业务意义的标签。例如：0 变换为“未开始”，1 变换为“正在修读”，2 变换为“已结课”。
- **逻辑计算：**系统根据变换逻辑自动处理学分授予。若成绩变换结果为“及格”，则自动将课程的总学分变换为该学生的“已获学分（`earned_credits`）”。

## 2.5 特征选择

如为了满足教务管理的安全性，系统实现了基于角色属性的数据过滤逻辑：

- **教师端过滤：**利用 Spring Security 获取当前登录教师的教工

号（teacher\_no），在执行数据查询请求时，动态将该编号注入 SQL 过滤条件，从而在海量数据中“选择”出仅属于该教师名下的学生名单。

- **敏感字段隐藏：**根据用户角色，在接口返回时有选择地屏蔽身份证号、联系方式等敏感特征，实现字段级的数据保护。

## 2.6 数据划分

系统根据业务生命周期对数据进行逻辑划分，主要依据 learn\_status 字段：

- **待修读数据集（Status 0）：**表示学生已选课但教学尚未激活，处于静态预备状态。
- **修读中数据集（Status 1）：**表示处于教学活跃期的数据，支持成绩录入操作。
- **归档数据集（Status 2）：**表示已完成评改的数据，该类数据转为只读状态，作为学生成绩档案长期保存。

# 第三章 函数与模型创建

## 3.1 函数的创建

本教务教学管理系统采用了面向对象的模块化开发思想。在业务逻辑层（Service 层），为了实现开课与成绩管理的核心闭环，系统设计了多个关键的业务处理函数。以下重点介绍软件开发中用到的核心业务函数，及其设计方法、参数说明。

### 1. 教师一键开课核心函数

- **函数名称：**startCourse
- **所属类：**BizTeacherGradeServiceImpl
- **设计方法：**该函数主要用于改变特定课程下所有选课学生的状态。在执行更新前，函数内置了业务防御机制，通过调用查询方法检查当

前课程的选课人数。若人数为零，则抛出异常中断操作；若验证通过，则调用数据访问层进行状态的批量更新。

- **参数说明：**
  - courseId (Long)：输入参数，代表需要开启教学任务的课程唯一标识 ID。
- **返回值：**int，代表数据库中受影响的行数（即成功转为“修读中”状态的学生数量）。

## 2. 成绩录入与学分结算综合函数

- **函数名称：**updateGrade
- **所属类：**BizTeacherGradeServiceImpl
- **设计方法：**该函数承担了成绩落库与学分判定的双重职责。函数首先对输入的分数进行边界值校验（0-100 分）。随后进入学分结算分支：若成绩达到及格线（60 分），则查询该课程的元数据，将总学分赋值给学生的“已获学分”属性；否则记为 0 学分。最后，将学生的修读状态强制置为“已结课”，并提交至数据库。
- **参数说明：**
  - studentCourse (BizStudentCourse)：输入参数，封装了包含主键 ID、录入分数（score）等信息的实体对象。
- **返回值：**int，表示更新操作的执行结果（1 为成功，0 为失败）。

## 3.2 模型的创建

在本系统的设计中，虽然未直接使用预测模型，但为了支撑复杂的教务流转，系统构建了高度抽象的领域实体数据模型与业务状态机模型。

### 1. 选课与成绩综合实体模型（BizStudentCourse）

- **设计方法：**该模型作为贯穿表示层、业务层与持久层的数据载体（DTO），不仅映射了中间表的基础物理字段，还通过属性扩展的方式，承载了多表 Join 联查产生的衍生数据。

- **核心参数说明：**

- id: 选课记录的唯一业务主键。
- score / earnedCredits: 成绩与已获学分，用于核心评价结算。
- studentName / college: 扩展属性，用于接收底层关联学生档案表（biz\_student\_profile）的投影数据，解决前端展示时的信息残缺问题。

## 2. 课程修读生命周期（状态机）模型

- **设计方法：**系统将学生修读一门课程的过程抽象为一个有限状态机（FSM）。通过 learn\_status 字段的严格流转，限制教师与学生在不同阶段的操作权限。

- **状态与流转参数说明：**

- **状态 0（未开始）：**初始状态。学生完成选课即进入此状态。在此状态下，系统屏蔽教师的打分接口。
- **状态 1（正在修读）：**活跃状态。由教师触发 startCourse 函数后从状态 0 跃迁至此。在此状态下，系统放开成绩录入权限。
- **状态 2（已结课）：**终态。由教师触发 updateGrade 函数后从状态 1 跃迁至此。到达此状态后，成绩数据被锁定并归档，禁止再次修改。

## 第四章 项目测试

介绍软件测试方法，并详细记录软件的自测过程。

### 4.1 测试方法

本系统的测试采用自底向上的集成测试与系统测试策略，综合运用了以下几种测试方法：

1. 黑盒测试（功能测试）：站在最终用户（教师/学生）的视角，验证系统的各项功能界面是否正常工作，交互逻辑是否符合预期（如：按钮点击响应、弹窗数据展示）。
2. 白盒测试（逻辑驱动测试）：针对核心业务 Service 层（如成绩校验、学分判定逻辑），进行边界值分析（如测试成绩录入 -1 分或 101 分的情况），验证异常抛出机制。
3. 接口联调测试（前后端集成）：利用浏览器的开发者工具（Network 面板），监控 Vue 前端发出的 Axios 请求与 Spring Boot 后端 API 接口的数据传输过程，重点排查状态码与 JSON 数据结构的匹配度。
4. 安全与权限测试：针对 Spring Security 框架，切换不同的角色账号登录，验证数据隔离能力和越权访问拦截能力。

### 4.2 测试过程

在系统的核心业务闭环（开课  $\rightarrow$  选课  $\rightarrow$  打分）集成测试中，我们设计了多个核心测试用例，并记录了测试中发现的问题及修复方案：

#### 测试用例 1：前端数据绑定与传输测试

- 测试动作：教师在“我的教学课程”列表中点击“学生名单与成绩管理”按钮。
- 预期结果：弹出当前课程的学生名单窗口，并根据课程 ID 查询出对应数据。
- 实际结果：控制台报错 请求参数类型不匹配：输入值为 undefined。
- 问题诊断与修复：经排查，Vue 页面向后端传参时误读了 row.courseId，而数据库表 biz\_course 的真实主键在 Java 实体类中映射为 id。将前端取值逻辑修正为 this.currentCourseId = row.id; 后，传参恢复正常。

#### 测试用例 2：MyBatis 连表查询结果映射测试

- 测试动作：打开某门课程的学生名单弹窗。
- 预期结果：表格中完整显示学生的学号、姓名、所属学院及当前状态。
- 实际结果：数据成功加载，但“姓名”和“所属学院”列显示为空白。
- 问题诊断与修复：经分析，底层 SQL 已正确执行 LEFT JOIN 并查询出了

姓名和学院，但业务中间表对应的 Java 实体类 (BizStudentCourse) 中遗漏了这两个扩展属性 (缺少对应的属性定义及 Getter/Setter)。导致 MyBatis 映射时数据丢失。在实体类中补充 studentName 和 college 字段并重启后端后，数据渲染成功。

#### 测试用例 3: RBAC 角色权限与越权拦截测试

- 测试动作: 学生账号登录系统, 点击“教务选课”菜单。
- 预期结果: 成功加载全校课程列表。
- 实际结果: 前端拦截器报错, 后端返回 HTTP 状态码 403 Forbidden。
- 问题诊断与修复: 证明系统安全机制生效。原因是该接口配有 @PreAuthorize 权限注解, 但未在后台管理系统中将 system:course:list 权限字符分配给学生角色。在管理员后台勾选对应菜单权限, 并让学生账号重新登录 (刷新 Redis 权限缓存) 后, 403 错误消除, 列表正常展示。同时发现, 若后端重启导致 Token 失效, 会返回 401 Unauthorized, 重新登录即可解决。

#### 测试用例 4: 业务规则防御测试

- 测试动作: 教师针对一门“无人选修”的课程, 点击“一键开启教学”。
- 预期结果: 系统拦截操作, 并提示无法开课。
- 实际结果: 前端捕获到 HTTP 500 错误, 后端抛出 ServiceException (“当前课程暂无学生选修, 无法开启教学任务”), 预期防御机制生效。

## 4.3 测试结论

经过系统性的功能自测、联调排错与安全验证, 得出以下测试结论:

1. 功能完整性: 系统的核心教务流转 (开课激活、实时选课、合规打分、学分结算) 均能按设计预期流转, 前端交互流畅, 业务状态机控制严谨。
2. 数据安全性: 基于 Spring Security 的鉴权机制和基于教工号/学号的数据过滤机制运行稳定, 有效阻断了水平越权与未授权访问。
3. 架构健壮性: 前后端分离架构中的 API 契约对接准确, 后端针对非法输入 (如分数越界、空课程开课) 具备完善的异常抛出与防御拦截能力。系统当前版本已达到设计要求, 业务逻辑闭环已经打通, 具备了上线部署和试运行的条件。

# 第五章 算法接口

## 5.1 接口代码

代码实现接口需要提供给软件人员, 用于后续功能的实现。

本系统采用 RESTful API 设计规范, 核心的开课与打分接口在前端封装为统一的请求方法, 供 Vue 组件异步调用。

## 1. 前端 API 请求封装代码 (JavaScript/Axios):

```
import request from '@/utils/request'

// 算法接口 1: 一键开启教学任务
export function startTeacherCourse(courseId) {
  return request({
    url: '/system/teacherGrade/start/' + courseId,
    method: 'put'
  })
}
```

```
// 算法接口 2: 录入成绩并执行学分结算
export function gradeTeacherStudent(data) {
  return request({
    url: '/system/teacherGrade/grade',
    method: 'put',
    data: data
  })
}
```

## 2. 后端 API 接收与调度代码 (Java/Spring Boot):

```
/**
 * 教师教学与成绩管理核心接口
 */

@RestController
@RequestMapping("/system/teacherGrade")

public class BizTeacherGradeController extends BaseController {

    @Autowired
    private IBizTeacherGradeService bizTeacherGradeService;
```

```
// 接收开课请求，触发状态机跃迁逻辑
@PreAuthorize("@ss.hasPermi('system:teacherGrade:edit')")
@Log(title = "一键开课", businessType = BusinessType.UPDATE)
@PutMapping("/start/{courseId}")
public AjaxResult startCourse(@PathVariable Long courseId) {
    return
toAjax(bizTeacherGradeService.startCourse(courseId));
}

// 接收打分请求，触发成绩合规校验与学分结算算法
@PreAuthorize("@ss.hasPermi('system:teacherGrade:edit')")
@Log(title = "录入成绩", businessType = BusinessType.UPDATE)
@PutMapping("/grade")
public AjaxResult gradeStudent(@RequestBody BizStudentCourse sc)
{
    return toAjax(bizTeacherGradeService.updateGrade(sc));
}
}
```

## 5.2 代码说明

根据系统架构设计，以上核心业务接口的具体说明如下：

### 1. 教学任务开启接口（startTeacherCourse）

- **前端接口文件名与位置：**命名为 teacherGrade.js，位于前端工程的 src/api/system/ 目录下。
- **后端控制器文件名与位置：**命名为 BizTeacherGradeController.java，位于后端工程的 ruoyi-system/src/main/java/com/ruoyi/system/controller/ 目录下。
- **输入数据格式与位置：**输入数据采用 RESTful 路径变量（Path

Variable) 的形式进行传递。前端直接将课程主键拼接在 URL 尾部 (例如: /system/teacherGrade/start/101), 后端通过 @PathVariable 提取 courseId 参数交由 Service 层处理。

## 2. 成绩录入与结算接口 (gradeTeacherStudent)

- **前端接口文件名与位置:** 命名为 teacherGrade.js, 位于前端工程的 src/api/system/ 目录下。
- **后端控制器文件名与位置:** 命名为 BizTeacherGradeController.java, 位于后端工程的 ruoyi-system/src/main/java/com/ruoyi/system/controller/ 目录下。
- **输入数据格式与位置:** 输入数据以 JSON 对象格式 (application/json) 通过 HTTP Request Body 传输。数据结构包含选课记录主键 id 以及教师录入的成绩 score。后端通过 @RequestBody 注解将该 JSON 数据反序列化为 BizStudentCourse 实体对象, 随后送入业务层的结算算法中计算是否及格并授予学分。

## 第六章 附件

### [1] 若依 (RuoYi) 全栈开源框架官方文档

- 参考位置：第一章 软件架构、第二章 数据过滤与权限设计。
- 说明：主要参考了其前后端分离架构的搭建规范、RBAC 角色权限控制模型以及 @PreAuthorize 权限注解的使用方法。

### [2] Spring Boot Reference Guide (官方参考指南)

- 参考位置：第三章 函数与模型创建、第五章 算法接口。
- 说明：主要参考了 RESTful API 的设计规范、Controller 层的请求映射（@GetMapping、@PostMapping 等）以及全局异常处理机制（ServiceException）。

### [3] Vue.js 2.x 官方中文文档 & Element UI 组件库开发手册

- 参考位置：第一章 软件架构、第四章 项目测试。
- 说明：主要参考了前端组件的生命周期、Axios 异步请求的 Promise 处理（.then().catch()），以及 <el-table>、<el-dialog> 等核心数据渲染组件的主子表交互设计。

### [4] MyBatis 3 官方参考文档

- 参考位置：第二章 数据集成（多表关联）、第三章 模型创建。
- 说明：主要参考了 XML 映射文件中的 LEFT JOIN 连表查询技术，以及 ResultType 与 Java 实体类（DTO）扩展字段的映射匹配规则。

### [5] MySQL 8.0 Reference Manual (官方参考手册)

- 参考位置：第二章 数据获取。
- 说明：主要参考了关系型数据库的表结构设计、主键与外键（student\_no）的关联约束规范。

### [6] 《软件工程：实践者的研究方法》（第 8 版）

- 参考位置：第一章 设计概述、第四章 项目测试。
- 说明：主要参考了面向对象的详细设计规范、模块化设计原则（高内聚低耦合）以及黑盒/白盒测试用例的设计方法。